项目完成报告：基于 MongoDB 的问卷系统（第一阶段）

项目仓库：https://github.com/Feiyu37yyc16/survey-system

一、项目目标

开发一个简化版在线问卷系统，功能类似问卷星或 Google Forms。注册用户可以创建问卷，通过链接分发问卷，其他人登录后填写，创建者查看统计结果。系统所有数据存储在 MongoDB 中，不使用关系数据库，不套用现成问卷框架。

第一阶段按初始需求完成系统设计与实现：系统设计、数据库设计、前后端程序、必要测试和项目文档，为第二阶段的需求变更打好基础。

二、系统设计

2.1 技术选型

部分	选型	理由
后端	Python 3 + Flask	轻量，路由和接口清晰，适合教学项目；代码量适中
数据库	MongoDB（pymongo）	题目、跳转规则、回答都是嵌套结构，文档模型天然匹配；统计用聚合管道
前端	原生 HTML/CSS/JS	需求不要求复杂界面，原生实现可读性好、无额外依赖
会话	Flask 签名 Cookie	登录状态由服务端签名维护，客户端只保存 Cookie

2.2 总体结构

浏览器（5 个静态页面 + fetch 调用接口）

|

▼

Flask 服务（app.py 路由层 + logic.py 业务逻辑层）

|

▼

MongoDB（users / surveys / responses 三个集合）

页面与功能对应：

- index.html：注册、登录
- dashboard.html：问卷列表、新建、发布、关闭、复制链接
- edit.html：编辑问卷（标题、说明、匿名、多次填写、截止时间、题目、跳转规则）
- fill.html：填写问卷（按顺序答题、自动跳转、错误提示、提交）
- stats.html：统计查看（整体 / 单题）

logic.py 里的校验、跳转、统计逻辑是纯函数，不依赖 Flask，方便单独测试。

2.3 功能实现情况

对照初始需求逐项实现：用户注册登录、只能看到自己创建的问卷；问卷创建、列表、发布、关闭、截止时间、短码链接；单选、多选、文本、数字四种题型及各自的限制条件；按单/多选/数字结果配置跳转；多人同时填写、匿名填写、多次填写；整体统计和单题统计。

三、MongoDB 设计

3.1 集合划分

三个集合，职责分离：

- users：用户账号（用户名、密码哈希、注册时间）
- surveys：问卷（标题、说明、状态、截止时间、短码、内嵌题目数组和跳转规则）
- responses：填写记录（survey_id、user_id、按 qid 保存的 answers 数组、提交时间）

3.2 关键设计决定

1. 题目内嵌在 surveys 里。题目只在问卷上下文存在，内嵌后一次查询拿到整份问卷的题目和规则，编辑和填写都不用二次查询或 join。

2. 一次填写一条 response，answers
数组只存实际回答过的题。被跳过的题不出现，统计时天然排除，统计口径和填写流程一致。

3. 跳转规则作为问卷数据存储，而不是写死在程序里。规则是 `{ condition: { op, value }, target }`，创建者可以在编辑页配置，程序只负责按规则执行。

4. 索引：username 唯一、url_code
唯一、owner_id、survey_id+submitted_at、survey_id+user_id。

5. 不冗余：responses 不存问卷标题和题目文本，统计按 survey_id 关联查询。

3.3 为什么不用关系数据库

题目数量不定、选项数量不定、限制条件随题型不同、跳转规则数量不定，这些在关系型里要拆成多张表靠 join 拼回来，或者退化成 JSON 字段。MongoDB 文档模型直接内嵌，代码里的结构和数据库里的结构一致。另外单选题分组计数、多选题展开数组计数、数字题求平均正好对应 `$unwind`、`$group`、`$avg`，统计在数据库内完成。

3.4 为什么适合 MongoDB

- 文档模型与业务对象一一对应，没有对象关系映射层；
- 问卷和回答的嵌套结构（题目数组、答案数组）不需要拆表；
- 聚合管道直接支持按选项分组、按数组元素展开的统计；
- 字段随需求可增可减（如多选的 min/max/exact），不需要预先设计完整 schema，为第二阶段需求变化留了余地。

四、跳转设计

4.1 规则模型

每道题可配多条跳转规则，规则条件按题型有对应操作符：

题型	操作符	含义
单选	eq	答案等于某选项
多选	contains	答案包含某选项
文本	eq / contains	等于 / 包含某文字
数字	gt / gte / lt / lte / eq / between	大于 / 大于等于 / 小于 / 小于等于 / 等于 / 落在区间

规则目标只能是本题之后的题，保存问卷时校验，防止循环跳转。

4.2 执行流程

填写页每答完一题，把答案和已答列表发给后端 answer 接口：

1. 校验当前答案；
2. 从当前题的规则里找第一条命中的；
3. 命中则从目标题起向后找第一道未答题，未命中则从当前题后一道起向后找；
4. 找不到返回 null，填写结束。

提交时后端把整份回答按同样规则模拟一遍，被跳过的题不要求回答，防止前端绕过校验。

4.3 示例

问卷共 6 题：性别 → 水果 → 职业 → 年龄 → 手机号 → 满意度。

- 选"女" → 跳过水果题直接到职业题；
- 多选含"苹果" → 跳过职业题直接到年龄题；
- 年龄 ≥ 18 → 跳过手机号直接到满意度题。

自动化测试覆盖了这三类跳转及无规则顺序推进两种分支，全部通过。

五、校验设计

校验分两层：编辑时的题目定义校验，填写时的答案校验。

题型	答案校验内容
单选	必须选择，且选项合法
多选	数量在最少/最多之间，或等于必须数；选项合法
文本	字数在最少/最多之间
数字	能转成数字；要求整数时必须为整数；在最小/最大值之间

非必答题允许留空，留空统一存
null，统计时排除。提交时再做一次全量校验（模拟跳转流程），保证必答题不漏答。

六、AI 使用过程

6.1 AI 使用日志

说明：本阶段开发全程使用 AI 编程工具协助。以下按作业要求记录"使用的工具、提示词、得到的代码、是否修改、修改了什么"。示例中的真实记录以实际使用过程为准，提交前请核对补充。

序号	工具	Prompt（提示词）	Result（得到的代码）	是否修改	修改了什么
1	AI 编程助手	帮我设计 MongoDB schema，用于问卷系统，包含用户、问卷、题目、跳转规则、填写记录	得到 users / surveys / responses 三个集合的字段设计，题目数组内嵌于问卷	修改	增加 allow_multi_submit、deadline 字段；跳转规则补上 between 操作符
2	AI 编程助手	用 Flask + pymongo 实现用户注册登录和问卷增删改查接口	得到 app.py 的路由框架和 werkzeug 密码哈希用法	修改	所有问卷接口加上 owner_id 权限判断；把业务校验抽到 logic.py
3	AI 编程助手	如何实现按答案跳转到指定题目，规则要能配置	得到跳转引擎思路：规则存问卷数据，答题时按条件计算下一题	修改	增加"跳转目标必须在本题之后"校验，防止循环；提交时全量模拟跳转
4	AI 编程助手	用 MongoDB 聚合管道统计单选、多选、数字题	得到 $unwind + $group + $avg 的聚合管道	修改	单选排除空值；多选先展开 value 再分组；文本题限制返回条数
5	AI 编程助手	生成一个简单的问卷编辑页和填写页	得到 HTML/JS 页面框架	修改	前端统一用公共 fetch 封装；错误提示与后端校验消息保持一致

6.2 AI 使用说明

AI 帮了什么：搭出了整体代码框架（Flask 路由、页面结构、聚合管道的写法）、提供了 pymongo 和 werkzeug 的用法，缩短了查文档的时间。

AI 做错了什么：生成的问题有——早期 schema
缺少多次填写和截止时间的字段；统计聚合没有考虑被跳过题和空答案；个别页面 JS
里重复赋值导致渲染异常；测试代码把匿名/截止时间参数写死，导致部分用例跑不过。

自己改了什么：补齐了业务校验（题目定义、答案、跳转目标合法性）；统一了接口返回格式和错误码；加了提交时的全量跳转模拟；空答案统一存 null
并从统计中排除；把测试辅助函数修正为透传问卷设置参数。

自己设计了什么：跳转规则的数据结构（condition + target）和"目标必须在题后"的防循环约束；问卷状态机（草稿/发布/关闭）与截止时间的统一检查；responses
只存实际回答过的题这一口径设计；非必答空值与统计排除的处理。

七、测试结果

自动化测试 37 条断言全部通过（tests/test_api.py，生成
tests/测试报告_自动化.md），手工测试 26 组步骤按预期执行。覆盖：注册登录、创建问卷、添加题目、跳转逻辑、答案校验、提交问卷、匿名填写、多次填写、截止时间、关闭、统计。

关键结论：

- 单选（选女跳 q3）、多选（含苹果跳 q4）、数字（≥18 跳 q6）三类跳转全部按配置执行；
- 被跳过的题不要求回答，也不计入统计；
- 必答题漏答、选项不合法、数量超限、文本长度、数字范围都能被正确拦截；
- 统计口径与填写路径一致（如被跳过的水果题只有 1 条路径的数据）。

八、遇到的问题与解决

1. 问题：非必答题留空时被校验拦截（空文本、空数字报错）。解决：答案校验开头放行非必答的空值；提交时空文本统一转成 null。

2. 问题：空答案会出现在统计里（单选的空选项计数、文本空行）。解决：提交时规范化空值为 null，单选题聚合管道排除 null；多选题空数组展开后天然不产生记录。

3. 问题：跳转目标可以配置成指向自身或前面的题，存在死循环风险。解决：保存问卷时校验目标必须存在且在本题之后，非法配置直接拒绝并提示。

4. 问题：测试辅助函数把匿名、多次填写、截止时间参数写死，导致匿名/截止时间相关用例失败。解决：辅助函数改为透传问卷设置参数，测试与真实场景一致。

5. 问题：问卷不可填（未发布/已关闭/已过期）在不同接口返回码不统一（400/404
混用）。解决：填写入口统一返回 404，前端统一处理。

6. 问题：多选题统计口径（选择次数合计）与单选题（回答人数）容易混淆。解决：在文档和统计返回里明确区分，并在报告中说明。

7. 问题：Flask 路由中 /api/surveys/code/<短码> 与 /api/surveys/<id>
可能冲突。解决：把带静态段 code 的路由注册在前面，避免被 id 路由吞掉。

九、总结

第一阶段按初始需求完成了问卷系统的设计与实现：三个集合的 MongoDB 设计、四种题型的校验、可配置的跳转引擎、完整的填写流程和聚合统计，自动化测试全部通过。设计中刻意保留了扩展空间（题目结构、问卷字段、跳转规则模型），为第二阶段根据需求变更做修改和扩展做好了准备。